Phoenix Multi-Channel Marketplace System Proposal

Date: 2026-06-17 Source PRD: `(PRD) Lotus's Marketplace System (TH).pdf`, v1.0.0, 2026-06-02 Role: Senior Solution Architect assessment

1. Executive Summary

Lotus's current Multi-channel Marketplace System is responsible for product master, price, promotion, stock, and order synchronization across marketplace channels and mart channels. The legacy design relies heavily on a single Microsoft SQL Server database and batch-oriented processing. This has become a bottleneck for large-scale synchronization:

- Marketplace channels: Shopee, Lazada, TikTok, Amaze, WeChat, Makro Pro.
- Mart channels: Shopee Mart, LINE MAN Mart, Grab Mart, Hato Mart.
- Approximate load:
  - 200,000 total SKUs.
  - Around 20,000 SKUs per marketplace channel.
  - Around 2,000 mart stores.
  - Around 10,000 SKUs per mart store.
  - Three-year peak-day forecast: 50,000 orders/day.
  - Short peak: 250 orders/second for no more than 2 minutes.

The proposed Phoenix Multi-Channel Marketplace System should replace the legacy batch-centric architecture with an event-driven, horizontally scalable platform using Go microservices, Kafka, Redis, highly available PostgreSQL with native partitioning, and React + Next.js for operations UI. Application-level PostgreSQL sharding is not required for the current three-year order forecast and should remain a future scaling option.

Phoenix does not replace WMS operations. It accepts seller orders, reserves stock, hands accepted orders to the existing fulfilment estate, consumes external status events, and synchronizes statuses back to seller channels. PO/BOL receiving, IBT workflow, picking, packing, printing, AWB generation, pallet/truck operations, ClickHouse, analytical data warehouses, Business Intelligence, and management reporting are explicitly out of scope. Operational SLA, queue, reconciliation, and campaign-readiness dashboards remain in scope.

The most urgent business problem is not only database cost. It is the inability to meet time-critical stock, price, and promotion SLAs, especially before double-date campaigns. The target architecture should therefore optimize for:

- Real-time or near-real-time ATS stock correctness.
- Delta-based product, price, and promotion sync.
- Channel-isolated scalability and failure containment.
- Fast recovery and replay through Kafka.
- Operational observability with clear SLA dashboards.

Recommended first quick win by end of August:

> Build a Phoenix Price and Promotion Delta Sync Pilot for the highest-impact channel group, backed by Kafka event flows, PostgreSQL sync ledger, and reverse-engineered channel adapters. Add a contained Redis ATS proof of concept in parallel. Keep old system as source/parallel-run fallback. This directly attacks midnight campaign SLA misses while proving the architecture needed to eliminate overselling in the next phase.

2. Current PRD Functional Baseline

The PRD defines the current Lotus's Marketplace System as an own-build WMS + OMS platform connected to RMS, OSIM, R10, ReSA, Sale Adaptor, API Gateway, Seller Centers, CMA, MKP Web, and Truck Management.

WMS functions in the current PRD — excluded from Phoenix scope

Function	Current behavior	Phoenix implication
1. RMS Product Master Sync	Daily full sync from RMS into PLU table, upsert per record, emit downstream refresh event.	Replace full-table dependency with product snapshot ingestion, delta detection, versioning, and event publication.
2. PO / BOL Receiving	WMS, OSIM, CMA, MKP DB, and printing flow for inbound receiving.	Out of scope. Phoenix consumes resulting stock changes from OSIM only.

Function	Current behavior	Phoenix implication
3. IBT Transfer	MKP creates IBT-OUT records, OSIM confirms, stock auto-sync follows.	Out of scope. Phoenix consumes resulting OSIM stock events without owning the IBT workflow.

OMS functions

Function	Current behavior	Phoenix implication
4. Get Product from Seller Center	Pull products every 4 hours and store SKU-platform parameters: Auto/Manual, price override, promotion override, share percentage, clubpack quantity.	Must be rebuilt as channel catalog ingestion and mapping service.
5. Get Order to Fulfilment	Poll orders, route to WMS, split tracking, ready-to-ship, receive AWB, capture sale, print, pallet, truck.	Phoenix owns order ingestion, reservation, external hand-off, and seller status sync only. All warehouse execution remains external.
6. Price and Promotion Sync	Read R10/LDD, map seller SKU, apply A/M mode, clubpack, promotion dates, compare previous price, call platform APIs only on change.	High-priority quick win. Delta sync can drastically shorten campaign readiness.
7. Stock Sync	Calculate available stock = base stock - sales - pending - unpaid - damage - flash reserve, allocate by share, sync to channels.	Highest-priority quick win. Replace with real-time ATS plus dynamic allocation.
8. Update Order Status and Capture Sale	Requires AWB before Auto POS capture, maps sale transaction to tracking code, retries failures.	Phoenix consumes the required external status/reference, performs idempotent capture where retained, and synchronizes seller status; AWB generation remains outside Phoenix.
9. Put-to-Pallet and Truck Management	Records truck, pallet, order, and tracking mapping; posts shipped status.	Out of scope.
10. Reports	Sales, platform, POS mapping, ready-to-ship, pallet, truck, location stock, stock by SKU.	BI and management reporting are out of scope. Only operational service-health and reconciliation views remain.

3. Target Architecture

Detailed engineering design, capacity model, runtime flows, deployment topology, and production-readiness criteria for the forecast of 50,000 orders/day and a 250 orders/second two-minute peak are defined in [`phoenix-target-architecture-250-ops.md`](phoenix-target-architecture-250-ops.md).

3.1 Architectural Principles

1. PostgreSQL is the business source of truth, not the work queue.
2. Kafka is the orchestration, replay, and shock absorber layer.
3. Redis is the real-time ATS and reservation engine, not the long-term ledger.
4. Channel adapters are isolated microservices with independent rate-limit handling.
5. Every write operation must be idempotent.
6. Every sync decision must leave an auditable ledger entry.
7. Old .NET channel adapters should be reverse-engineered to reduce domain discovery and API behavior risk, but not copied as-is.
8. Seller delivery is planned against 100 requests/minute per confirmed quota scope, with only 80 requests/minute used normally and 20 reserved for urgent work and retries.
9. Bulk updates support up to 100 items/request as an optimistic ceiling, not a guarantee; effective batch size is discovered, configured, and monitored per operation.

3.2 Proposed Services

Domain	Service	Main responsibility
Integration	`rms-ingestion-service`	Pull or receive RMS product snapshots, compute deltas, publish product events.
Integration	`r10-price-promo-service`	Consume LDD/R10 price and promotion data, compute effective price, publish delta events.
Integration	`osim-stock-service`	Consume OSIM stock events/snapshots, compute ERP delta, publish stock movements.
Inventory	`ats-service`	Maintain Redis ATS, reservations, Lua atomic update scripts, stock ledger writes.
Inventory	`allocation-service`	Dynamic channel allocation using ATS, safety stock, channel priority, sales velocity, and rate-limit health.
Catalog	`catalog-mapping-service`	Own PLU, seller SKU mapping, Auto/Manual mode, clubpack, override settings.
Sync	`sync-orchestrator-service`	Convert product, price, promo, and stock deltas into channel sync commands.
Channel	`channel-adapter-*`	One adapter per platform or platform family with shared distributed rate limiting, dynamic batch sizing, priority scheduling, retries, and quota-drain forecasting.
Orders	`order-ingestion-service`	Poll or receive orders, normalize statuses, publish order events.
Orders	`reservation-service`	Deduct Redis ATS on order creation/payment and release on cancel/expire.

Domain	Service	Main responsibility
Orders	`order-handoff-service`	Deliver accepted orders to the existing fulfilment estate through an idempotent asynchronous contract.
Orders	`external-status-ingestion-service`	Consume external order-status events and publish canonical seller-facing transitions.
Sales	`capture-sale-service`	Integrate with Sale Adaptor/Auto POS with idempotent capture.
Operations	`admin-api`	BFF/API for Next.js admin UI.
UI	`admin-web`	Next.js dashboards and operations screens.
Observability	`sync-monitoring-service`	SLA, queue, DLQ, adapter health, campaign readiness dashboards.

3.3 Data Stores

Store	Recommended use
PostgreSQL	Product master, SKU mapping, channel config, price/promo effective state, stock ledger, sync ledger, order state, idempotency keys.
PostgreSQL native partitioning	Partition large ledgers by date and domain. Do not introduce application-level sharding at the current forecast; retain stable keys and interfaces so it remains a future option.
Existing OMS order partitions	Reuse `phoenix-oms-mkp-service`: monthly `orders`, `order_items`, `order_status_history`, and `packages` partitions; non-partitioned `order_refs`/`package_refs`; idempotent three-month-ahead partition creation; guarded archive tooling.
Redis HA / cluster-ready	Real-time ATS and reservations on an HA replication group; separate disposable cache/rate-limit deployment; cluster-compatible keys for future horizontal scaling.
Kafka	Product, price, promo, stock, allocation, sync command, sync result, order, reservation, capture-sale events.
Object storage	Raw RMS/R10/OSIM snapshots, adapter request/response archive, and reconciliation evidence.
OpenSearch	Optional application and platform log search with short retention. It is not a BI store or source of transactional truth.

3.4 Kafka Topic Model

Topic	Purpose
`rms.product.snapshot.received`	Raw product snapshot metadata.
`product.delta.detected`	Insert/update/delete/status changes by SKU.
`r10.price-promo.snapshot.received`	Raw price/promo batch metadata.
`price-promo.delta.detected`	Effective price or promotion changes only.
`osim.stock.snapshot.received`	Stock snapshot metadata from OSIM.
`stock.delta.detected`	ERP stock movement/delta per SKU/store.
`stock.ats.updated`	Redis ATS result after atomic merge.
`allocation.rebalanced`	Channel/store allocation output.
`sync.command.created`	Platform-specific update command.
`sync.result.received`	Platform response, retry, failure, DLQ state.
`order.received`	Normalized order event.
`stock.reservation.created`	ATS deduction for order reservation.
`stock.reservation.released`	Release on cancel/expiry/failure.
`sale.capture.requested`	Auto POS capture request.
`sale.capture.completed`	Capture result and sale transaction mapping.

3.5 Four-Phase Journey to Precision

Phase A: Delta Integration with ERP

The legacy pattern treats incoming source snapshots as final truth. Phoenix should compare the new snapshot to the last accepted version and publish only the relative delta.

Example:

- Previous RMS/OSIM state: 100.
- New source state: 150.
- Phoenix computes delta: +50.
- Phoenix applies +50 against real-time ATS, not as an overwrite.

This eliminates stale overwrite, especially when platform sales have already deducted ATS after the source snapshot was created.

Phase B: Real-Time ATS Synchronization

Redis maintains live ATS per SKU, store, and channel allocation pool. Lua scripts make stock movements atomic:

1. Read current ATS.
2. Apply ERP delta.
3. Apply active reservations/order deductions.
4. Enforce floor at zero.
5. Write updated ATS and publish `stock.ats.updated`.

This prevents race conditions between stock receipt, sales, cancellation, damage, and platform sync.

Phase C: Dynamic Allocation Engine

Replace fixed `%Share` as the only mechanism. Keep `%Share` as a safety/config baseline, then add dynamic weighting:

- Rolling 15-minute, 1-hour, and 24-hour sales velocity.
- Platform conversion and cancellation rate.
- Campaign priority.
- Channel API health and rate-limit backlog.
- Safety stock and minimum display stock.

For example, if TikTok is selling faster than Lazada during a campaign, allocation shifts more ATS to TikTok automatically while maintaining a protected minimum for other channels.

Phase D: Omni-Channel Delivery

Kafka fans out optimized updates to channel adapters. Each adapter handles:

- Platform authentication.
- Request shape and protocol.
- Rate limits.
- Dynamic bulk size; never assume the advertised maximum of 100 items is accepted by every endpoint.
- Retry and backoff.
- Platform-specific validation.
- Dead-letter classification.
- Response normalization.

This prevents one slow or failing marketplace API from blocking all other channels.

4. Migration Strategy

Phoenix should be introduced as a strangler replacement, not a big-bang rewrite. The old system continues to run while Phoenix takes ownership feature by feature.

Recommended migration slices

1. Observability wrapper over current system.
2. Stock and price delta pilot for selected channels.
3. Full stock ATS and allocation rollout.
4. Full price and promotion campaign SLA rollout.
5. Product mapping and catalog sync.
6. Order reservation, external fulfilment hand-off, and seller status synchronization.
7. Decommission only the legacy synchronization workloads replaced by Phoenix.

WMS operations, BI/reporting, and analytical warehouse migration are not part of the strangler plan.

Reverse engineering old .NET adapters

The old .NET source code should reduce effort in these areas:

- Platform authentication flows.
- Existing API endpoint coverage.
- Field mapping and validation rules.

- Known platform error codes.
- Rate-limit behavior and retry heuristics.
- Edge cases such as clubpack, manual mode, and platform SKU differences.

Expected effort reduction:

- 25% to 40% reduction for known marketplace adapters with clean source code.
- 10% to 20% reduction if code is poorly structured or platform APIs have changed significantly.
- Minimal reduction for new mart-scale store logic if the old adapters were not designed for store-level fanout.

The reverse engineering deliverable should be an adapter behavior spec and contract tests, not a line-by-line port.

5. First Milestone: Quick Win by End of August

5.1 Calendar and Capacity Assumptions

Current date: 2026-06-17. Target: 2026-08-31.

Approximate working time: 52 business days.

Squad:

- 4 developers.
- 1 QA.
- 1 Squad Lead / Tech Lead with people-management responsibility and hands-on system design capability.

Nominal capacity:

- Developers: 4 x 52 = 208 developer-days.
- QA: 1 x 52 = 52 QA-days.
- Squad Lead / Tech Lead: 52 lead-days.

Practical delivery capacity after ceremonies, alignment, environment setup, dependency waiting, and production hardening:

- Developers: around 145 to 165 effective developer-days.
- QA: around 35 to 42 effective QA-days.
- Squad Lead / Tech Lead: around 40 to 46 effective lead-days, typically split into 18 to 24 hands-on architecture/engineering days and 18 to 22 delivery, stakeholder, review, and people-management days.

5.2 Milestone Option A: Stock ATS Quick Win

Goal: Reduce overselling by making stock deduction and ERP stock delta application near real-time for one or two high-volume channels.

Scope:

- OSIM stock ingestion or stock snapshot import.
- Redis ATS model and Lua atomic delta merge.
- Order reservation ingestion from selected channels.
- Allocation baseline using current `%Share` plus safety stock.
- Sync command generation for selected channels.
- One or two channel adapters reverse-engineered from .NET.
- Operations dashboard for ATS, sync latency, retry, DLQ.

Estimated effort:

Workstream	Mandays
Discovery and .NET adapter reverse engineering	12
Architecture, event contracts, data model	10
Kafka topic setup and common event libraries	12
PostgreSQL schema and sync ledger	10
Redis ATS and Lua scripts	18
OSIM stock delta ingestion	14
Reservation/order deduction integration	16
Allocation baseline	10

Workstream	Mandays
Channel adapter 1	14
Channel adapter 2	12
Admin dashboard and operational views	12
QA automation and test data	18
Performance and failure testing	10
UAT, parallel run, release hardening	12
Total	**180 mandays**

Fit for August: Borderline but possible if limited to one channel plus read-only comparison for the second. Full two-channel production may be risky.

Impact:

- Directly reduces overselling.
- Establishes core Phoenix architecture.
- Demonstrates real-time stock correctness.

Complexity:

- High, because order reservation and stock correctness require strong idempotency and careful reconciliation.

5.3 Milestone Option B: Price and Promotion Delta Sync Quick Win

Goal: Meet midnight SLA for campaign eligibility by replacing long batch sync with delta-based price/promotion sync for priority channels.

Scope:

- R10/LDD ingestion.
- Price/promotion effective-state calculation.
- Clubpack and Auto/Manual rules.
- Delta compare against last synced state.
- Sync command fanout through Kafka.
- Reverse-engineered adapters for Shopee, Lazada, TikTok, or the top two channels.
- Campaign readiness dashboard.
- Legacy fallback and parallel result comparison.

Estimated effort:

Workstream	Mandays
Discovery and .NET adapter reverse engineering	12
Architecture, event contracts, data model	8
R10/LDD ingestion	12
Price/promotion calculation engine	16
Delta detection and sync ledger	12
Kafka sync command flow	8
Channel adapter 1	14
Channel adapter 2	12
Channel adapter 3, limited scope	10
Campaign readiness dashboard	12
QA automation and reconciliation reports	18
Performance testing with 200k SKU dataset	12
UAT, parallel run, release hardening	12
Total	**158 mandays**

Fit for August: Good if scope is top two or three channels and product creation is out of scope.

Impact:

- Directly addresses 4 PM / 6 PM batch pain and midnight SLA.
- High business visibility for double-date campaigns.
- Lower correctness risk than stock ATS.

Complexity:

- Medium-high, mostly platform API and rate-limit handling.

5.4 Milestone Option C: Observability and Legacy Acceleration Layer

Goal: Improve current system without taking over writes immediately.

Scope:

- CDC or query-based extraction from legacy SQL Server.
- Kafka event mirror.
- Sync latency dashboard.
- Failure classification.
- Pre-campaign readiness report.
- Adapter queue depth and retry visibility.
- Optional worker acceleration for one sync path.

Estimated effort:

Workstream	Mandays
Legacy database analysis	10
CDC/query extractor	16
Kafka mirror topics	8
Sync audit model	8
Dashboard and alerting	18
Failure classification	10
Optional worker acceleration	20
QA and reconciliation	16
UAT and rollout	10
Total	**116 mandays**

Fit for August: Very good.

Impact:

- Improves visibility quickly.
- Does not fully solve overselling or SLA misses.
- Useful as foundation but may be seen as insufficient business impact.

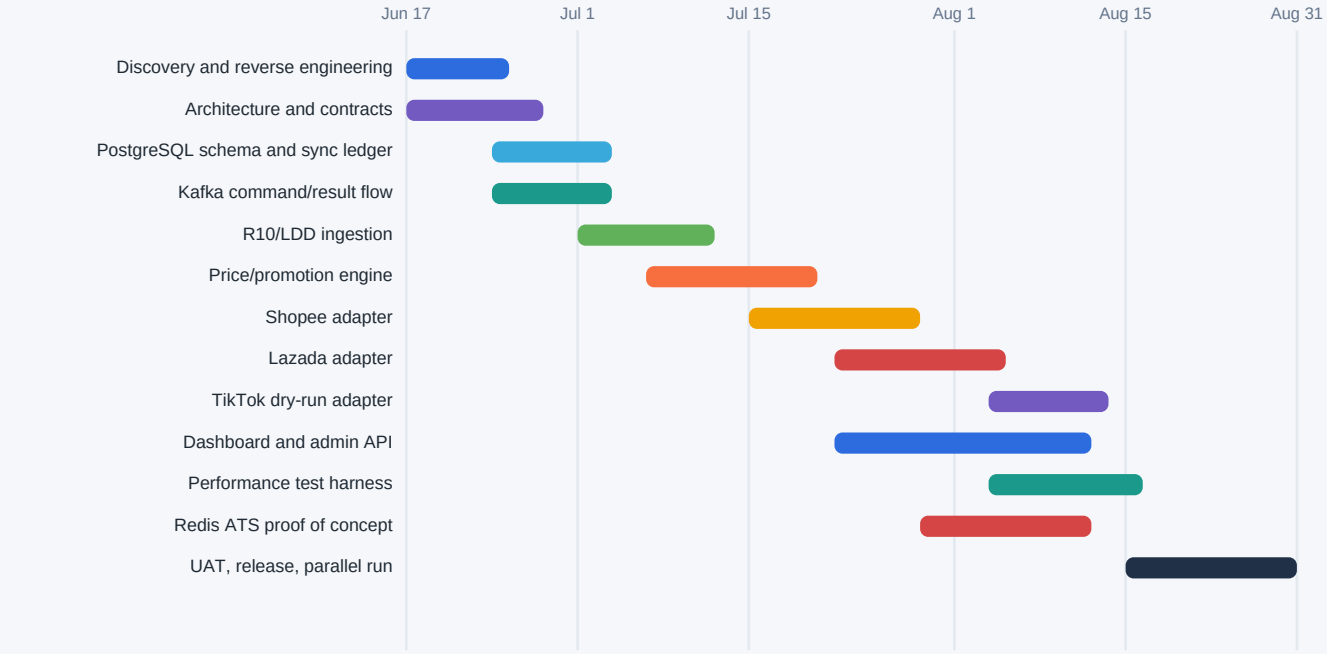
Complexity:

- Medium.

5.5 Milestone Recommendation

5.5 Milestone Recommendation - Gantt view

Illustrative working-day plan from 2026-06-17 to 2026-08-31 for Option B plus Redis ATS POC.



Note: dates are planning placeholders; final sequencing should be confirmed after the one-week .NET adapter/API audit.

Recommended: **Option B plus a small ATS proof of concept from Option A**.

Why:

- Price and promotion delta sync has the best August delivery fit and directly addresses the midnight double-date campaign SLA.
- It proves Kafka, PostgreSQL sync ledger, channel adapters, dashboards, retry, and DLQ patterns.
- It avoids the highest-risk stock correctness cutover during the first milestone.
- Add a contained Redis ATS proof of concept for one SKU/store/channel flow to prove the future stock engine without making it the production path yet.

Recommended August scope:

Deliverable	Target
Price/promotion delta engine	Production pilot
Channels	Shopee + Lazada production pilot, TikTok dry-run or limited pilot
Dataset	All mapped Auto SKUs for selected channels, or top campaign SKUs if API limits are tight
SLA	Campaign delta sync completed within 30 minutes in pilot, stretch target under 5 minutes for changed SKUs
Dashboard	Campaign readiness, pending sync count, failed sync, DLQ, channel API success rate
Redis ATS	Technical proof of concept only, one channel flow, not full production cutover
Legacy fallback	Old system remains fallback for production safety

Estimated August milestone effort:

Workstream	Dev MD	QA MD	Tech Lead MD
Discovery and reverse engineering	10	2	5
Architecture and contracts	5	1	6
PostgreSQL schemas and sync ledger	7	2	3
Kafka command/result flow	7	2	3
R10/LDD ingestion	12	3	3
Price/promotion engine	15	5	4
Channel adapter: Shopee	13	5	2
Channel adapter: Lazada	11	5	2
TikTok dry-run adapter	8	3	1
Dashboard and admin API	12	4	2
Performance test harness	7	6	3
UAT, release, parallel run	7	8	8
Redis ATS proof of concept	9	2	3

Workstream	Dev MD	QA MD	Tech Lead MD
Total	**123**	**48**	**45**

This fits the available developer capacity if the squad is protected from unrelated work. The Tech Lead materially reduces risk by owning event contracts, adapter design reviews, reconciliation design, and critical production-readiness decisions. QA capacity remains tight; reduce TikTok to dry-run and automate reconciliation early.

6. Full Roadmap and Effort Estimate

Effort is estimated in mandays for phase planning, then converted to manhours for feature planning using 1 manday = 8 manhours. Estimates include engineering, QA support, and technical delivery work, but exclude procurement, infrastructure approval delays, marketplace certification delays, and vendor waiting time.

6.1 Phase Summary in Mandays

Phase	Name	Scope	Mandays
0	Discovery and Foundation	Reverse engineer .NET adapters, platform API contract matrix, architecture, environments, CI/CD, observability baseline.	175
1	August Quick Win	Price/promo delta sync pilot, Shopee/Lazada production pilot, TikTok dry-run, campaign dashboard, Redis ATS POC.	203
2	Real-Time Stock and Allocation	OSIM delta ingestion, Redis ATS production, reservations, allocation engine, stock sync rollout to marketplace channels.	420
3	Catalog and Product Sync	RMS product delta, seller product ingestion, mapping UI, Auto/Manual config, clubpack, product sync to channels.	300
4	Order Lifecycle and External Hand-off	Order ingestion, reservation lifecycle, external fulfilment hand-off, capture sale, external status ingestion, seller status updates.	393
5	Mart Scale Rollout	Store-level stock fanout for Shopee Mart, LINE MAN Mart, Grab Mart, Hato Mart; rate-limit optimization; native partitioning.	460
6	WMS and Operations	Explicitly out of scope.	0
7	Operational Optimization	Predictive allocation, anomaly detection, campaign simulator, and auto-remediation; no BI or reporting data mart.	145
8	Scoped Legacy Decommission	Data migration, reconciliation, cutover, retirement of replaced synchronization workloads, runbook handover.	220
Total			**2,316 mandays**

At a stable squad of 4 developers plus 1 Squad Lead / Tech Lead, the scoped replacement is not realistically a single-year effort unless scope is reduced or more squads are added. With 3 cross-functional squads using the same lead-led model, major replacement can be achieved in approximately 8 to 11 months after foundation, subject to external API constraints and existing fulfilment integration dependencies.

6.2 Feature-Level Effort in Manhours

Phase 0: Discovery and Foundation

Feature	Manhours
Current system walkthrough and PRD validation	160
.NET adapter reverse engineering for existing channels	240
Marketplace and mart API contract matrix	160
Domain event model and API standards	160
Local/dev/test environment setup	160
CI/CD, container build, deployment baseline	120
Observability baseline: logs, traces, metrics	160
Security baseline: OAuth, mTLS pattern, Vault integration design	120
Test strategy and synthetic data generator design	120
Total	**1,400 hours**

Phase 1: August Quick Win

Feature	Manhours
R10/LDD ingestion	120
Price/promotion effective-state engine	160
Clubpack, Auto/Manual, promotion active rules	80

Feature	Manhours
Delta detection and last-synced state ledger	120
Kafka sync command and result topics	80
PostgreSQL schema and partitioned sync ledger	80
Shopee price/promo adapter	140
Lazada price/promo adapter	120
TikTok dry-run adapter	80
Retry, backoff, DLQ handling	80
Campaign readiness dashboard	120
Performance test for 200k SKU and changed-SKU batches	100
Parallel-run reconciliation against legacy output	100
Redis ATS proof of concept	120
UAT, release, runbook	120
Total	**1,620 hours**

Phase 2: Real-Time Stock and Allocation

Feature	Manhours
OSIM stock snapshot and delta ingestion	240
Stock ledger and reconciliation model	200
Redis HA and cluster-ready ATS key model	160
Lua atomic delta merge scripts	200
Order reservation and release API	240
Pending/unpaid/damage/flash reserve integration	240
Allocation baseline using current `%Share`	160
Dynamic allocation using sales velocity	240
Stock sync adapters for marketplace channels	480
Rate-limit aware scheduler	200
Stock dashboard and oversell monitor	160
Load, chaos, and recovery testing	320
Production rollout and reconciliation	320
Total	**3,360 hours**

Phase 3: Catalog and Product Sync

Feature	Manhours
RMS product snapshot ingestion	160
Product delta detection and versioning	200
PLU/product master schema	120
Seller Center product pull per channel	320
Seller SKU mapping service	200
Product parameter UI: Auto/Manual, share, clubpack, overrides	280
Product sync command generation	160
Product adapter functions per priority channel	360
Validation, deduplication, inactive SKU handling	160
Product reconciliation dashboard	160
QA and UAT	280
Total	**2,400 hours**

Phase 4: Order Lifecycle and External Hand-off

The monthly PostgreSQL order-partition design is already implemented in `phoenix-oms-mkp-service` and is the required baseline, not a new architecture work item. Phase 4 must gap-assess and harden the existing schema, partition controller, reference-table integrity, default-partition monitoring, archive/restore runbooks, and partition-aware repository queries. The phase estimate remains unchanged until that implementation gap assessment is completed.

Feature	Manhours
Order ingestion per marketplace channel	480

Feature	Manhours
Order normalization and idempotency	200
Reservation lifecycle integration with ATS	240
Existing fulfilment hand-off contract	160
Sale Adaptor/Auto POS capture	260
Sale transaction to tracking mapping	160
Retry, escalation, and operations queue	200
External status event ingestion and normalization	160
Order status sync back to platforms	240
Test automation and channel simulations	480
Parallel run, reconciliation, rollout	560
Total	**3,140 hours**

Phase 5: Mart Scale Rollout

Feature	Manhours
Store-SKU data model and native partitioning strategy	240
Mart store hierarchy and channel config	200
Shopee Mart adapter scaling	260
LINE MAN Mart adapter scaling	260
Grab Mart adapter scaling	260
Hato Mart adapter scaling	220
Bulk stock fanout scheduler	320
Store-level rate-limit and priority queue	240
Store-level dashboard and exception queue	200
Data compression and cache optimization	160
Load testing with 2,000 stores x 10,000 SKUs	400
Rollout and operational handover	320
Total	**3,680 hours**

Phase 6: WMS and Operations — Out of Scope

No Phoenix engineering effort is allocated to receiving, IBT, warehouse screens, printing, picking/packing, AWB generation, pallet, truck, or other WMS operations. Phoenix integrates only through accepted-order hand-off and external status/stock events.

Phase 7: Operational Optimization

Feature	Manhours
Predictive allocation model	280
Campaign readiness simulator	240
Anomaly detection for stock and price	240
Auto-remediation playbooks	200
QA and stakeholder validation	200
Total	**1,160 hours**

Phase 8: Scoped Legacy Decommission

Feature	Manhours
Legacy data retention and migration plan	160
Reconciliation framework	240
Feature-by-feature cutover runbooks	200
Retirement of replaced synchronization workloads	240
Archive old adapter services	120
Disaster recovery and rollback rehearsal	240
Security review and access cleanup	120
Final UAT and business signoff	240
Knowledge transfer and operations handover	200
Total	**1,760 hours**

7. Key Non-Functional Targets

Capability	PRD target	Phoenix target
RMS product sync	Daily, within 30 minutes	Delta publication within 15 minutes after source availability; full reconciliation daily.
Product pull from seller center	Every 4 hours	Every 1 hour for active channels, configurable.
Stock allocation + sync	Within 5 minutes	p95 under 60 seconds for changed stock, campaign top SKUs under 15 seconds where platform API allows.
Price and promotion sync	Within 5 minutes	Changed SKU p95 under 5 minutes when the changed set fits the certified quota/batch envelope; otherwise show forecast completion and campaign risk.
Seller outbound capacity	Not defined	Plan at 80 of 100 requests/minute per confirmed quota scope; reserve 20 for retry/urgent work; bulk maximum 100 items/request but effective size is runtime evidence.
Order acceptance to external hand-off	p95 under 1 minute	Move from polling to webhook where platforms support it; measure external fulfilment time separately.
Capture sale	p95 under 5 seconds	Keep p95 under 5 seconds with idempotent retry.
API success rate	99%	99.5% platform-adjusted, with clear exclusion for platform outage.
Oversell prevention	Not met today	Target near-zero oversell caused by internal stock latency; platform delay tracked separately.

8. Risks and Mitigations

Risk	Impact	Mitigation
Platform API rate limits or smaller-than-advertised bulk sizes block fast sync	Miss campaign SLA	Shared 80/20 quota budget, dynamic batch sizing, priority queues, changed-SKU-only sync, drain-time forecast, dry-run capacity tests, and quota negotiation.
Old .NET adapter source is incomplete or outdated	Reverse engineering benefit lower than expected	Start with source audit in week 1; produce adapter spec and contract tests before implementation.
OSIM/R10/RMS source data arrives late	Phoenix cannot meet end-to-end SLA alone	Dashboard source arrival time separately from Phoenix processing time.
Redis ATS inconsistency after outage	Stock correctness risk	Write stock ledger to PostgreSQL, replay Kafka events, scheduled reconciliation jobs.
Mart scale is much larger than marketplace scale	Performance and cost risk	Separate mart fanout architecture, store-level partitioning, priority queues, and phased rollout.
Big-bang replacement creates operational risk	Business disruption	Use strangler migration, parallel run, feature flags, and rollback per channel/function.
QA bottleneck	August scope risk	Automate reconciliation tests early, reduce TikTok to dry-run, use synthetic load generator.

9. Fancy Later-Phase Features

These features are not required to fix the pain points, but they can help position Phoenix as a strategic commerce platform rather than a technical rewrite.

Feature	Value
Campaign Command Center	Shows each double-date campaign's product eligibility, price readiness, stock readiness, platform sync backlog, and risk score.
Predictive Stock Allocation	Uses sales velocity, channel conversion, campaign calendar, and margin to allocate ATS before demand spikes.
SLA Autopilot	Automatically pauses low-priority syncs and prioritizes campaign SKUs when midnight SLA risk rises.
Oversell Root-Cause Explorer	Links an oversold order to stock event, reservation, platform sync attempt, API response, and timing gap.
Digital Twin Sync Simulator	Runs "what if" scenarios for 200k SKUs and 2,000 mart stores before campaign go-live.
Adapter Certification Lab	Contract tests and platform sandbox tests that certify channel adapters before deployment.
AI Operations Assistant	Summarizes sync failures, suggests remediation, and drafts incident updates.
Smart Promotion Guardrails	Detects suspicious price drops, expired promotions, missing seller SKU mapping, or invalid clubpack pricing before sync.
Channel Health-Based Allocation	Reduces allocation to channels with API outage, high cancellation, or slow fulfilment and reallocates to healthy channels.

10. Recommended Next Steps

1. Confirm August quick-win scope: price/promotion production pilot for Shopee and Lazada, TikTok dry-run, Redis ATS proof of concept.

2. Start one-week .NET adapter source audit and platform API contract review.
3. Build synthetic 200k-SKU and campaign-delta test dataset.
4. Lock event contracts and sync ledger schema before implementation starts.
5. Set up Kafka, PostgreSQL, Redis, observability, and deployment baseline.
6. Agree cutover rules: old system remains fallback, Phoenix pilot writes only for selected channel/SKU scope.
7. Define campaign SLA acceptance test: source data arrival time, Phoenix processing time, platform accepted time, and failed SKU remediation time.

11. Acceptance Criteria for August Milestone

Area	Acceptance criteria
Price/promo delta	Only changed Auto SKUs generate sync commands; Manual SKUs are skipped with reason.
Clubpack	Clubpack price calculation matches legacy output for sampled SKUs.
Promotion	Active and expired promotion decisions match R10/LDD rules.
Channel adapters	Shopee and Lazada pilot adapters can send updates, handle retries, and record platform responses.
Performance	200k SKU scan completes and changed-SKU command generation stays within agreed SLA.
Seller quota	All adapter replicas combined stay within 100 requests/minute; batch sizes 1/20/50/100 and 429 retry behavior are tested; dashboard reports effective items/minute and drain time.
Campaign readiness	Dashboard shows total SKUs, eligible SKUs, pending sync, failed sync, DLQ, and platform acceptance status.
Reconciliation	Phoenix pilot output can be compared against legacy output by SKU, channel, price, promotion, timestamp, and status.
Fallback	Operations can disable Phoenix writes and return selected scope to legacy sync.
ATS POC	Redis Lua script demonstrates atomic ERP delta + live deduction merge with deterministic replay test.